

Multi-Class Text Classification: A Comparative Analysis of TF-IDF and Skip-gram Models

Zarif Rashid, Rafia Zarin Alisha
Department of Computer Science and Engineering
Course: Natural Language Processing II
Student IDs: 23201317, 23201425
Group No: 08

Abstract—This project presents a comparative study of multi-class text classification on an assigned news headline dataset. The task contains four categories: Business, Science and Technology, Sports, and World News. The work follows the required pipeline of exploratory data analysis, three preprocessing versions, two word representation techniques, one classical machine learning model, and several neural network models. TF-IDF was used with Logistic Regression and a Deep Neural Network, while Skip-gram Word2Vec was used with SimpleRNN, GRU, LSTM, Bidirectional SimpleRNN, Bidirectional GRU, and Bidirectional LSTM. Hyperparameters were tuned using validation macro F1-score, and the official test set was used only for final evaluation. The best individual model was No preprocessing + TF-IDF + Deep Neural Network with 0.9165 accuracy and 0.9163 macro F1-score. However, optimum preprocessing achieved the best average macro F1-score across all models, showing that EDA-based cleaning gave the most stable overall performance.

Index Terms—Text classification, TF-IDF, Skip-gram, Word2Vec, Logistic Regression, Deep Neural Network, RNN, GRU, LSTM.

I. INTRODUCTION

Text classification is a central task in Natural Language Processing (NLP), where a model assigns a text document to one of several predefined classes. This project focuses on multi-class classification of news headlines. The goal is not only to build one model, but also to compare how different preprocessing strategies, word representations, and model architectures affect final performance.

The project is motivated by the fact that news categories often contain strong class-specific keywords. For example, Business headlines frequently contain words related to markets, prices, companies, and oil; Sports headlines contain words related to games, teams, wins, and seasons; Science and Technology headlines contain words such as software, Microsoft, and internet; and World News headlines contain words such as Iraq, president, and minister. Therefore, both traditional sparse features such as TF-IDF and dense distributed features such as Skip-gram Word2Vec were investigated.

The required workflow was followed carefully. The training data was used for training and validation, while the official testing data was used only for final evaluation. Three text versions were prepared: no preprocessing, extreme preprocessing, and optimum preprocessing. The models were compared using accuracy, macro F1-score, confusion matrices, and classification reports.

II. METHODOLOGY

A. Dataset and Exploratory Data Analysis

The assigned dataset contains news headlines with their corresponding news topics. The training file contains 93,151 rows and two columns: *News Headline* and *News Topic*; the official test file contains 12,000 rows. Four classes are present: Business, Science and Technology, Sports, and World News. The training data had no missing values, but the EDA revealed heavy wrapper noise, duplicate headlines, source tokens, digits, and entity artifacts. These observations were used directly when designing the extreme and optimum preprocessing versions.

TABLE I: Main EDA findings from the training data.

EDA item	Value
Training samples	93,151
Number of classes	4
Missing values	0
Exact duplicate rows	16,378
Normalized duplicate headline groups	10,635
Conflicting duplicate headline groups	20
Average raw word count	46.77
Average cleaned word count	37.66
Average sentence count	1.88
Texts containing HTML tags	100.00%
Texts containing URLs	2.15%
Texts containing digits	58.31%
Texts containing entity/source artifacts	30.58%
Texts containing AP/Reuters/AFP tokens	19.93%
Unique tokens after EDA cleaning	54,993
Rare tokens appearing once	16,007

Table I shows that the raw text is not clean enough to use without analysis. Every row contains HTML tags and special characters because the headlines include wrappers such as `<html>`, `<body>`, `<br>`, and `<b>`. Digits appear in 58.31% of the samples, so they were removed in the extreme version but treated carefully during EDA because numbers can carry news information such as stock values, years, scores, and product versions. The duplicate analysis also found 20 conflicting normalized duplicate groups, which means some nearly identical headlines appear with different labels. Fig. 1 visualizes the main noisy patterns and confirms why HTML, symbol, entity, URL, and source-token cleaning were necessary.

The class distribution in Table II and Fig. 2 shows that the training set is imbalanced. Business is the largest class, while Sports is the smallest class, with an imbalance ratio of

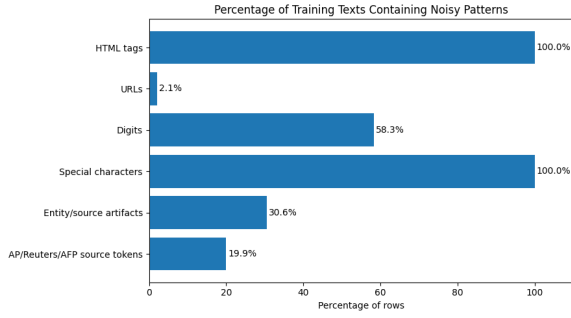


Fig. 1: Noisy pattern analysis of the training texts. HTML tags and special characters appear in every row, while digits, entity artifacts, and news-source tokens are also frequent.

TABLE II: Class distribution in the original training data.

Class	Count	Percentage
Business	36,316	38.99%
Science and Technology	22,422	24.07%
World News	19,118	20.52%
Sports	15,295	16.42%

about 2.37:1. Therefore, macro F1-score was used along with accuracy so that minority-class performance was not hidden by the larger classes.



Fig. 2: Class distribution in the original training data. Business is the largest class and Sports is the smallest class.

The length analysis showed that the average raw headline entry has 46.77 words, but after removing wrappers for EDA, the average cleaned length becomes 37.66 words. The class-wise average cleaned word counts were close: World News had 38.80 words, Sports 37.68, Business 37.39, and Science and Technology 37.09. This means the classes are not separable by length alone; vocabulary and phrase patterns are more important. Fig. 3 shows the cleaned length distribution and class-wise boxplots.

TABLE III: Class-wise cleaned length summary from EDA.

Class	Mean words	Median	Max
World News	38.80	39	145
Sports	37.68	37	124
Business	37.39	37	134
Science and Technology	37.09	36	157

Vocabulary analysis gave clearer modeling signals. Fig. 4 shows that terms such as oil, company, microsoft, software, iraq, president, game, and season are strong topic indicators. However, source tokens such as reuters, ap, and afp also appear frequently, so the optimum preprocessing removed repeated source artifacts while

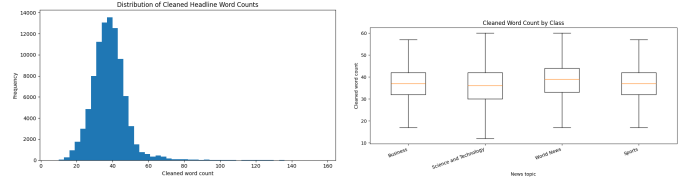


Fig. 3: Length-based EDA. Most cleaned headlines are concentrated around the mid-30 to mid-40 word range, and the class-wise boxplots are similar, so length alone is not a strong separator.

preserving meaningful short tokens such as us, uk, eu, un, and ai.

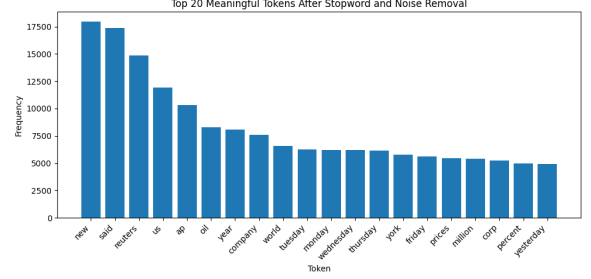


Fig. 4: Top meaningful words after stopword and noise removal. The vocabulary contains useful class-specific terms as well as repeated news-source tokens.

The n-gram analysis in Fig. 5 also supported TF-IDF with unigram and bigram features. Some frequent bigrams are topic-relevant, such as oil prices, united states, prime minister, wall street, and president bush. Other bigrams such as ap ap, reuters reuters, and afp afp confirmed the need to remove repeated source tokens in the cleaned datasets.

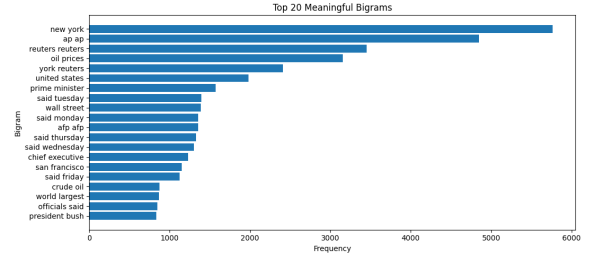


Fig. 5: Top bigrams from the EDA. Topic phrases support unigram-bigram TF-IDF, while repeated source bigrams justify source-token cleaning.

Fig. 6 provides an overall view of the cleaned training vocabulary. The word cloud confirms that the dataset contains strong lexical signals, which explains why TF-IDF based models performed strongly in the final experiments.

B. Preprocessing

Three versions of the dataset were created.

No preprocessing: The original text was kept unchanged. This version was included because the project required one dataset without preprocessing before TF-IDF or embedding conversion.

Extreme preprocessing: This version applied strong cleaning. Dataset wrapper text was removed, text was lowercased, punctuation and numbers were removed, stopwords were removed, very short useless tokens were removed, EDA-based noise words were removed, and Porter stemming was applied.

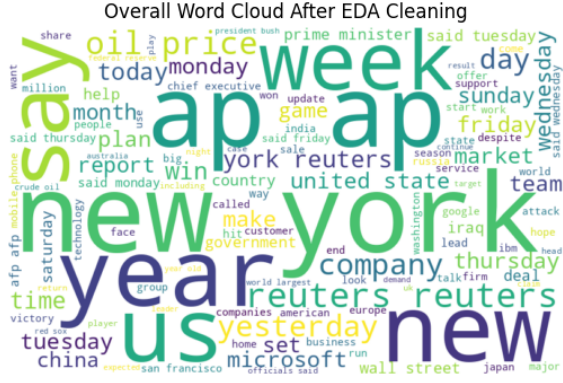


Fig. 6: Overall word cloud after EDA cleaning. The dominant terms show the importance of topic-specific vocabulary.

This reduced vocabulary size and produced compact keyword-like features.

Optimum preprocessing: This version was based on the EDA findings in Fig. 4 and Fig. 5. Dataset wrapper text and source/noise tokens were removed, text was lowercased, punctuation and numbers were removed, stopwords were removed, and very short useless tokens were removed. Important short words such as us, uk, eu, un, ai, tv, and pc were kept. Lemmatization and stemming were not used in the optimum version, so the text became cleaner while preserving original class-related keywords. Additionally, these normalization techniques were avoided in the optimum version because news headlines contain important proper nouns, acronyms, and domain terms, and aggressive normalization could distort useful signals such as company names, locations, and technology/product terms.

C. Word Representation Techniques

Two word representation techniques were used: TF-IDF for Logistic Regression and the Deep Neural Network, and Skip-gram Word2Vec for all recurrent neural network models.

TF-IDF: Term Frequency-Inverse Document Frequency (TF-IDF) is a sparse representation that gives higher weight to words that are frequent in a document but less frequent across the full corpus [1]. This is useful for news classification because class labels often depend on discriminative words and phrases. The term frequency, inverse document frequency, and final weight were written as

$$tf(w, d) = \log(1 + f(w, d)), \quad (1)$$

$$idf(w, D) = \log \left(\frac{N}{f(w, D)} \right), \quad (2)$$

$$tfidf(w, d, D) = tf(w, d) \times idf(w, D), \quad (3)$$

where $f(w, d)$ is the frequency of word w in document d , $f(w, D)$ is the number of documents in corpus D that contain w , and N is the total number of documents. TF-IDF was tuned using different maximum feature sizes and n-gram ranges.

Skip-gram Word2Vec: Skip-gram Word2Vec learns dense word embeddings by predicting surrounding context words from a center word [2]. Words that appear in similar contexts

receive nearby vector representations, so the model can capture semantic similarity better than raw word counts. In this project, Skip-gram embeddings were trained on the project training text using 100 dimensions, context window 5, minimum count 1, and 15 training epochs. The trained vectors were placed into an embedding matrix and used by the recurrent models with a tokenizer limit of 10,000 words and sequence length 60.

D. Models and Architectures

One classical machine learning model and seven neural network models were trained for each preprocessing version. Fig. 7 shows the architecture of the best-performing model: the TF-IDF based Deep Neural Network.

Logistic Regression: Logistic Regression was used as the classical baseline with TF-IDF vectors. It estimates a linear decision boundary over the sparse TF-IDF feature space and predicts the class using a softmax-style probability distribution. This model is strong for text classification because many news categories are separable by discriminative keywords. Balanced class weights were applied to reduce the effect of training class imbalance, and the tuned hyperparameters were max_features, ngram_range, and C.

Deep Neural Network: The DNN used TF-IDF vectors as input. The architecture contained three hidden Dense layers, Dropout after each hidden layer, and a final softmax layer for four-class classification. The first hidden layer always had at least 128 neurons, satisfying the project requirement. Balanced class weights and early stopping were used to improve generalization.

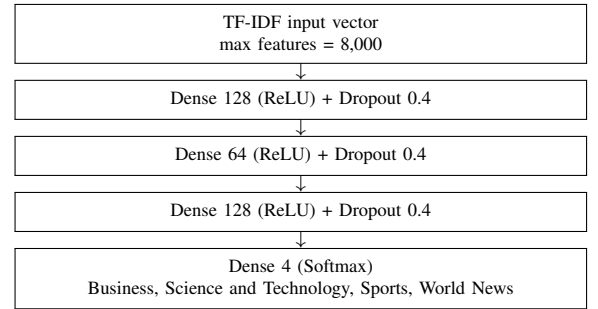


Fig. 7: Architecture of the best-performing model: No preprocessing + TF-IDF + Deep Neural Network.

SimpleRNN: SimpleRNN was used as the basic recurrent model for Skip-gram sequences. It processes tokens in order and updates a hidden state at each timestep. This makes it useful for testing whether a simple sequential model can capture headline patterns. However, SimpleRNN can struggle with long-term dependencies and vanishing gradients, which explains its weaker performance in the final results.

GRU: The Gated Recurrent Unit (GRU) model extends a simple recurrent network with update and reset gates. These gates help the model keep useful information and discard less useful information while reading the headline sequence. GRU is often more stable than SimpleRNN because it handles sequence memory more effectively. In this project, GRU provided one of the strongest Skip-gram based results.

LSTM: The Long Short-Term Memory (LSTM) model uses input, forget, and output gates to control information flow through a memory cell. This design helps reduce the vanishing-gradient problem and allows the model to retain important sequence information. LSTM was tested to compare a more expressive recurrent architecture against SimpleRNN and GRU. Its performance was generally competitive, although not better than the best GRU and bidirectional models.

Bidirectional SimpleRNN: Bidirectional SimpleRNN reads the input sequence in both forward and backward directions. This allows the representation of a word to use context from both sides of the headline. The architecture was included to test whether bidirectional context could improve the simpler recurrent model. It improved over the weakest SimpleRNN settings, but it still lagged behind gated bidirectional models.

Bidirectional GRU: Bidirectional GRU combines forward and backward GRU layers. It benefits from both gating and two-directional context, which is useful for short news headlines where important clues can appear anywhere in the sequence. This model achieved the strongest or near-strongest Skip-gram performance for the cleaned datasets. Its results indicate that bidirectional gated memory is effective for sequence-based headline classification.

Bidirectional LSTM: Bidirectional LSTM combines forward and backward LSTM layers. It captures context from both directions while using LSTM memory cells to preserve important information. This makes it a strong sequence model for Skip-gram embeddings. In the final comparison, Bidirectional LSTM performed close to Bidirectional GRU and clearly better than SimpleRNN.

III. HYPERPARAMETER TUNING

Manual random search was used from predefined search spaces. The possible hyperparameter values were manually selected, and a small number of combinations were randomly sampled and evaluated using validation macro F1-score. The official test set was not used to choose hyperparameters.

Hyperparameter search space.

Component	Values considered
TF-IDF + Logistic Regression	max_features: 3,000, 5,000, 8,000; ngram_range: (1,1), (1,2); C: 0.5, 1.0, 2.0
TF-IDF + DNN	Dense layers with first layer ≥ 128 neurons; dropout 0.3–0.5; learning rate 0.001 or 0.0005; batch size 64 or 128; epochs 10 or 15
Skip-gram sequence models	64 or 128 recurrent units considered; dropout 0.3–0.5; learning rate 0.001 or 0.0005; batch size 64 or 128; epochs 10 or 15

For TF-IDF + Logistic Regression, different values of max_features were tested to control vocabulary size. Unigram and unigram-bigram features were tested because some news topics are represented by single words such as “oil”, “game”, and “microsoft”, while others are better captured by phrases such as “stock market”, “prime minister”, and “world cup”. Different C values were tested to control regularization.

For the DNN, different numbers of neurons, dropout rates, learning rates, batch sizes, and epochs were tested. The first layer was kept at 128 neurons or more because the project

TABLE IV: Best validation settings for TF-IDF models.

Prep.	Model	Selected setting	Val. F1
None	LR	8k features; unigram; $C = 0.5$	0.9080
Extreme	LR	8k features; 1–2 grams; $C = 0.5$	0.9023
Optimum	LR	8k features; unigram; $C = 0.5$	0.9037
None	DNN	128–64–128; drop 0.4; lr .001	0.9132
Extreme	DNN	128–64–128; drop 0.4; lr .001	0.9082
Optimum	DNN	128–64–128; drop 0.4; lr .001	0.9073

specification required at least 128 neurons in the first layer. Dropout was used to reduce overfitting, while learning rate, batch size, and epochs were selected based on validation macro F1-score.

For the Skip-gram based RNN, GRU, LSTM, and bidirectional models, the number of units, dropout rate, learning rate, batch size, and epochs were tuned. In the final executed notebook run, the logged sampled configurations for the sequence models used 64 recurrent units, which gave stable validation performance. The best validation settings for each sequence model are summarized below.

Best validation settings for Skip-gram sequence models.

Preprocessing	Units	Selected setting	Val. F1 range
No preprocessing	64	drop 0.3; lr .0005–.001; 10 epochs	0.7004–0.8714
Extreme preprocessing	64	drop 0.3–0.5; lr .0005–.001; 10–15 epochs	0.8497–0.9014
Optimum preprocessing	64	drop 0.3–0.5; lr .0005–.001; 10–15 epochs	0.8681–0.9020

Hyperparameters were selected using validation macro F1-score instead of test accuracy. This avoided using the official test set during model selection. Since the task is multi-class classification and the training data has some class imbalance, macro F1-score was used as the main validation metric because it gives equal importance to all classes.

IV. RESULTS AND DISCUSSION

A. Overall Results

Table V shows the final test performance of all 24 experiments. The best individual experiment was No preprocessing + TF-IDF + Deep Neural Network, which achieved 0.9165 accuracy and 0.9163 macro F1-score. The worst experiment was No preprocessing + Skip-gram + SimpleRNN, which achieved 0.7076 accuracy and 0.7112 macro F1-score. In Table V, green bold values mark the best macro F1-score within each preprocessing version, and red bold values mark the worst macro F1-score within each preprocessing version.

B. Preprocessing Comparison

Although the best individual model used no preprocessing, the best average preprocessing strategy was the optimum preprocessing version. The average macro F1-score of optimum preprocessing was 0.8943, compared with 0.8901 for extreme preprocessing and 0.8503 for no preprocessing. This trend is shown in Fig. 8. The result suggests that EDA-based cleaning made model performance more stable across TF-IDF and Skip-gram models.

No preprocessing performed strongly for TF-IDF models because TF-IDF naturally reduces the impact of common words and can still capture discriminative raw tokens. However, no preprocessing was weaker for Skip-gram sequence models because repeated wrappers and noisy tokens affected

TABLE V: Final test results for all experiments. Best and worst rows are highlighted within each preprocessing group using macro F1-score.

Preprocessing	Representation	Model	Accuracy	Macro F1
No preprocessing	TF-IDF	Deep Neural Network	0.9165	0.9163
No preprocessing	TF-IDF	Logistic Regression	0.9123	0.9120
No preprocessing	Skip-gram	GRU	0.8684	0.8688
No preprocessing	Skip-gram	Bidirectional GRU	0.8688	0.8685
No preprocessing	Skip-gram	Bidirectional LSTM	0.8649	0.8646
No preprocessing	Skip-gram	LSTM	0.8628	0.8633
No preprocessing	Skip-gram	Bidirectional SimpleRNN	0.7939	0.7974
No preprocessing	Skip-gram	SimpleRNN	0.7076	0.7112
Extreme preprocessing	TF-IDF	Deep Neural Network	0.9113	0.9112
Extreme preprocessing	TF-IDF	Logistic Regression	0.9065	0.9063
Extreme preprocessing	Skip-gram	Bidirectional GRU	0.9008	0.9008
Extreme preprocessing	Skip-gram	Bidirectional LSTM	0.9004	0.9006
Extreme preprocessing	Skip-gram	GRU	0.8960	0.8956
Extreme preprocessing	Skip-gram	Bidirectional SimpleRNN	0.8833	0.8831
Extreme preprocessing	Skip-gram	LSTM	0.8736	0.8728
Extreme preprocessing	Skip-gram	SimpleRNN	0.8513	0.8506
Optimum preprocessing	TF-IDF	Deep Neural Network	0.9110	0.9109
Optimum preprocessing	TF-IDF	Logistic Regression	0.9053	0.9050
Optimum preprocessing	Skip-gram	Bidirectional LSTM	0.8988	0.8988
Optimum preprocessing	Skip-gram	Bidirectional GRU	0.8983	0.8985
Optimum preprocessing	Skip-gram	GRU	0.8973	0.8972
Optimum preprocessing	Skip-gram	LSTM	0.8908	0.8903
Optimum preprocessing	Skip-gram	Bidirectional SimpleRNN	0.8845	0.8847
Optimum preprocessing	Skip-gram	SimpleRNN	0.8686	0.8689

the learned embedding and sequence representation. Extreme preprocessing also performed well because it removed stop-words and used stemming, which reduced vocabulary size and helped keyword-based classification. However, stemming can distort original words, while the optimum version preserved original word forms.

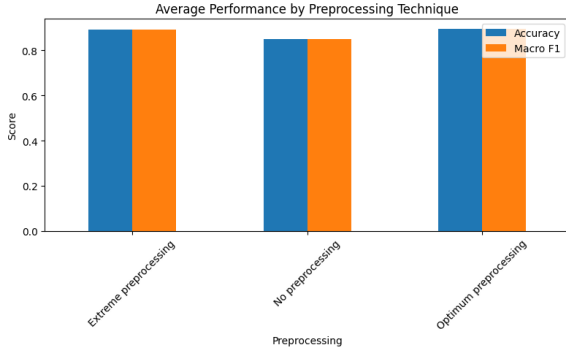


Fig. 8: Average accuracy and macro F1-score by preprocessing technique. This figure supports the preprocessing comparison.

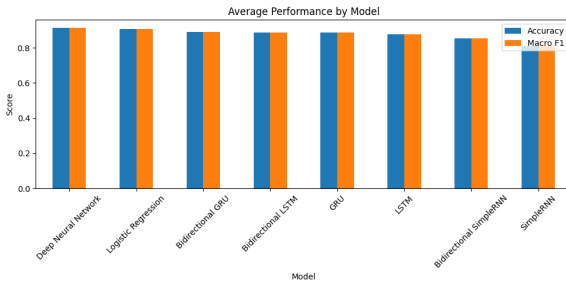


Fig. 9: Average accuracy and macro F1-score by model. This figure supports the model comparison.

C. Model Comparison

The DNN had the best average performance, with average macro F1-score of 0.9128. It performed best because its dense

hidden layers could learn non-linear combinations of TF-IDF keyword and phrase features, helping it capture interactions between related terms that are common in short news headlines. Logistic Regression was second with 0.9078 macro F1-score. These values are shown in Fig. 9 and confirm that TF-IDF was highly effective for this dataset. Logistic Regression performed strongly because TF-IDF produces sparse high-dimensional keyword features, and news classes are often linearly separable using such terms.

Among the Skip-gram sequence models, Bidirectional GRU and Bidirectional LSTM performed best. This is expected because bidirectional models read sequences in both directions, and GRU/LSTM units handle sequence memory better than SimpleRNN. SimpleRNN had the lowest average score because it has weaker memory and is more affected by vanishing gradient problems.

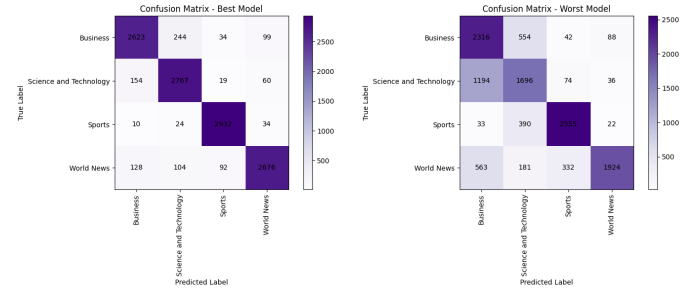


Fig. 10: Confusion matrices shown together: best model on the left (No preprocessing + TF-IDF + DNN) and worst model on the right (No preprocessing + Skip-gram + SimpleRNN).

D. Evaluation Metrics

Accuracy and macro F1-score were reported for every model in Table V. Classification reports were generated to show precision, recall, and F1-score for each class. Confusion matrices were generated for all 24 experiments; Fig. 10 shows

the best and worst cases together for direct comparison. Accuracy was useful because the official test set was balanced across four classes. However, macro F1-score was used as the main comparison metric because it treats every class equally and is more reliable when the training distribution is imbalanced.

V. CONCLUSION

This project compared TF-IDF and Skip-gram word representations across classical machine learning and neural network models for multi-class news headline classification. The best individual result was obtained by No preprocessing + TF-IDF + Deep Neural Network, with 0.9165 accuracy and 0.9163 macro F1-score. Logistic Regression with TF-IDF also performed very strongly, confirming that sparse keyword-based features are effective for this dataset.

The optimum preprocessing strategy achieved the best average macro F1-score across all models. This shows that EDA-based cleaning helped remove dataset artifacts while preserving useful news-topic keywords. Skip-gram sequence models performed reasonably well, especially Bidirectional GRU and Bidirectional LSTM, but they did not outperform TF-IDF-based models. A likely reason is that the Skip-gram embeddings were trained only on the project dataset, while TF-IDF directly captured discriminative class-specific words.

Limitations include the small number of hyperparameter trials, the use of dataset-trained Word2Vec instead of large pre-trained embeddings, and the absence of optional transformer-based models. Future work could include more extensive hyperparameter tuning, pretrained embeddings, transformer

models such as BERT, and additional error analysis of misclassified examples.

REFERENCES

- [1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [2] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," arXiv:1301.3781, 2013.
- [3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [4] J. L. Elman, "Finding structure in time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [5] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014, pp. 1724–1734.
- [6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [8] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [9] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009.
- [10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [11] M. Abadi *et al.*, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2016.
- [12] Ř. Rehurek and P. Sojka, "Software framework for topic modelling with large corpora," in *Proc. LREC Workshop on New Challenges for NLP Frameworks*, 2010, pp. 45–50.
- [13] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conference*, 2010, pp. 56–61.